Datenstrukturen

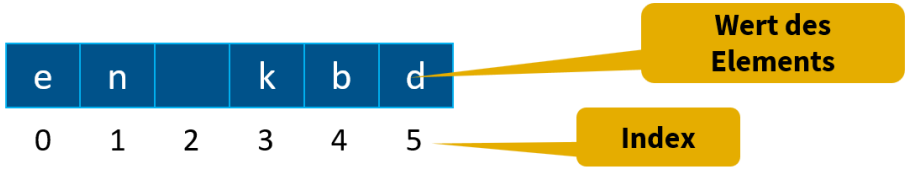
Relevante Aspekte für LF11

Array

Definition: Eine Struktur mit fester Länge (Größe muss bei Erstellung festgelegt werden).

Hauptvorteil: Sehr schneller Zugriff auf Elemente über den Index.

Nachteil: Unflexibel in der Größe; Einfügen/Löschen ist aufwendig, da Elemente ggf. verschoben werden müssen, sonst entstehen leere Felder.



ArrayList

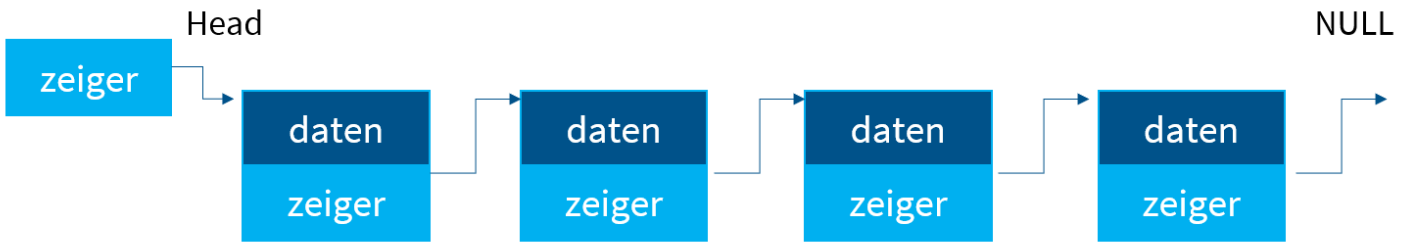
Definition: Im Gegensatz zum Standard-Array ist sie dynamisch erweiterbar.

Merkmale: Sie wächst automatisch, wenn mehr Platz benötigt wird, behält aber den schnellen Index-Zugriff bei.

Linked List (Verkettete Liste)

Definition: Eine dynamische Struktur, bei der jedes Element (Knoten) auf das nächste verweist.

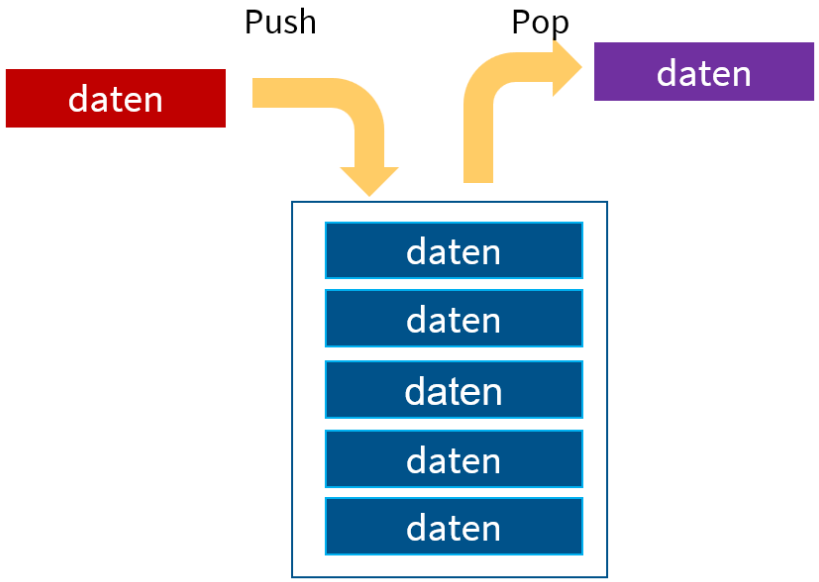
Stärke: Besonders schnelles Einfügen und Löschen (vor allem am Anfang), da lediglich Zeiger angepasst werden müssen und keine Elemente verschoben werden.



Stack (Stapel)

- **Prinzip:** *LIFO* (Last In – First Out). Das Element, das als letztes auf den Stapel gelegt wurde, wird als erstes wieder entnommen.

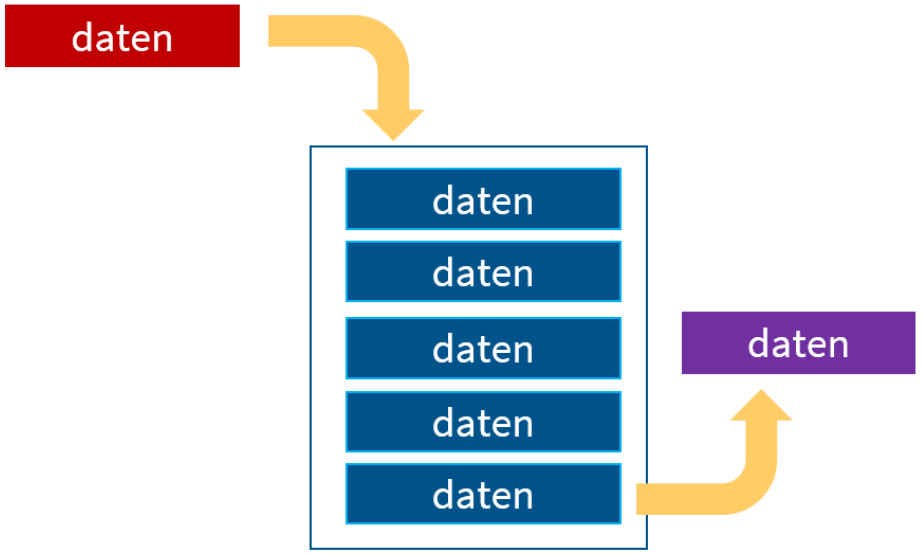
- **Visualisierung:** Wie ein Stapel Teller – man nimmt immer den obersten zuerst weg.



Queue (Warteschlange)

Prinzip: *FIFO* (First In – First Out). Wer zuerst kommt, wird zuerst bedient.

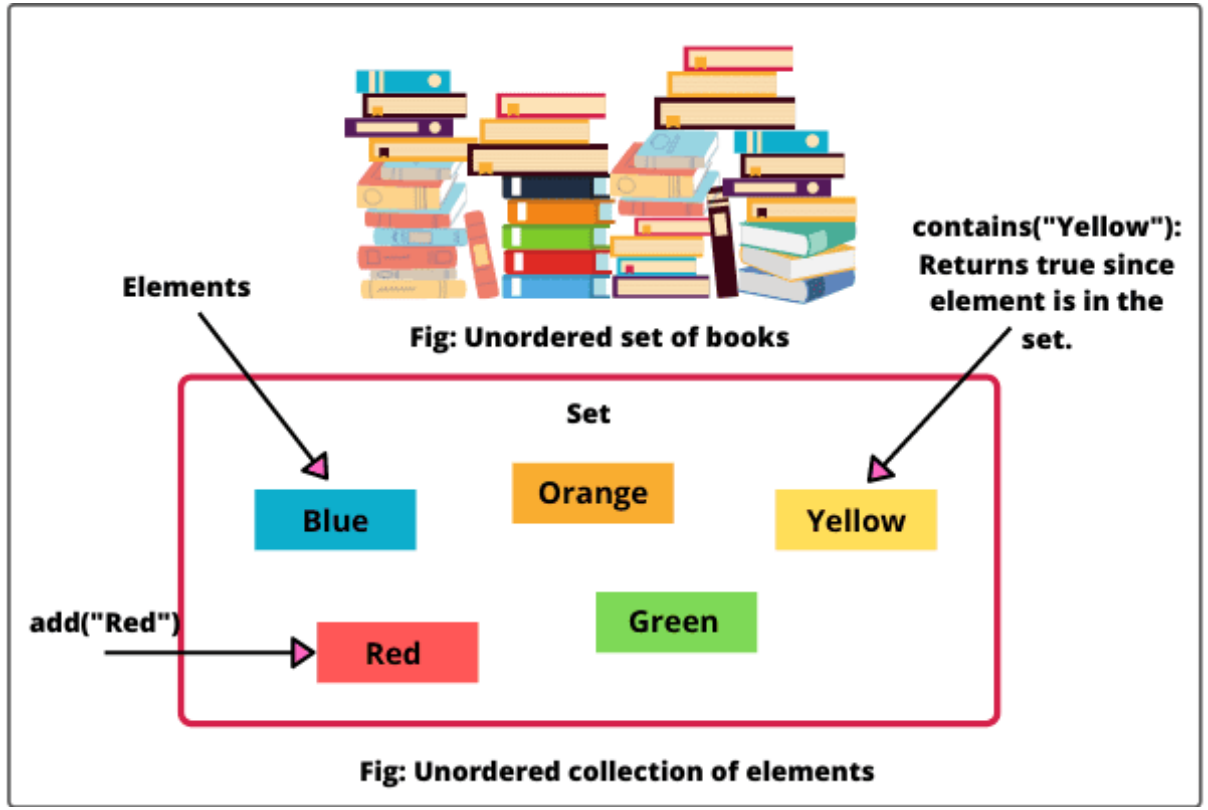
Anwendung: Typische Puffer-Systeme oder Druckwarteschlangen.



Set (Menge)

Definition: Eine ungeordnete Sammlung von einmaligen Elementen.

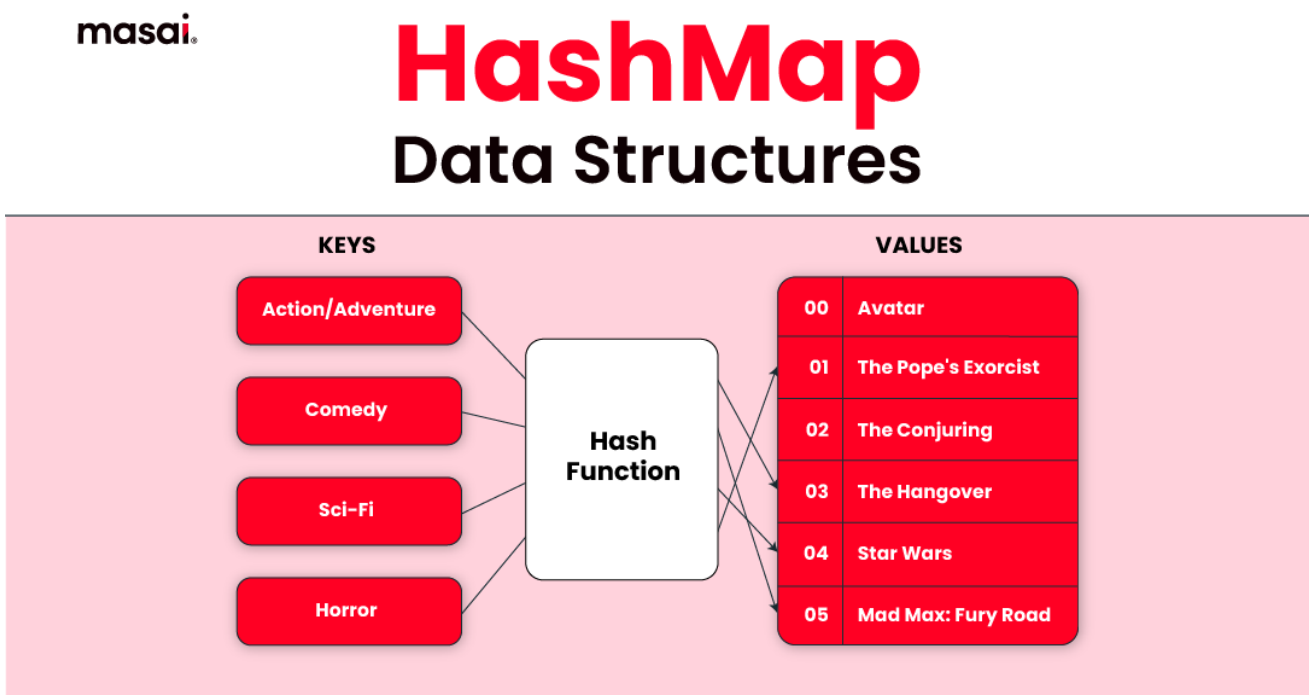
Wichtig: Ein Set erlaubt keine Duplikate. Wenn man ein Element doppelt hinzufügen möchte, wird dies ignoriert.



Bildquelle: Set in Java - Scientech Easy

Hashbasierte Strukturen

Grundprinzip ist das **Hashing**: Ein "Hash-Algorithmus" berechnet aus einem Schlüssel eine Adresse (Index) im Speicher. So weiß das Programm sofort, wo die Daten liegen, ohne suchen zu müssen.



<u>Beispiele in Java:</u>

HashMap, HashSet

Bildquelle: Understanding HashMap Data Structure With Examples

Merkmale & Vorteile hashbasierter DS

Schnelligkeit:
Der Zugriff erfolgt in nahezu konstanter Zeit (O(1)), völlig unabhängig von der Datengröße.

Schlüssel-Wert-Logik:
Ideal zum schnellen Wiederfinden von Daten anhand eines eindeutigen Identifikators (z. B. Matrikelnummer -> Studentendaten).

Nachteile & Risiken hashbasierter DS

Keine Sortierung:
Die Daten liegen „irgendwie“ im Speicher. Es gibt keine garantierte Reihenfolge (anders als beim Array oder Baum).

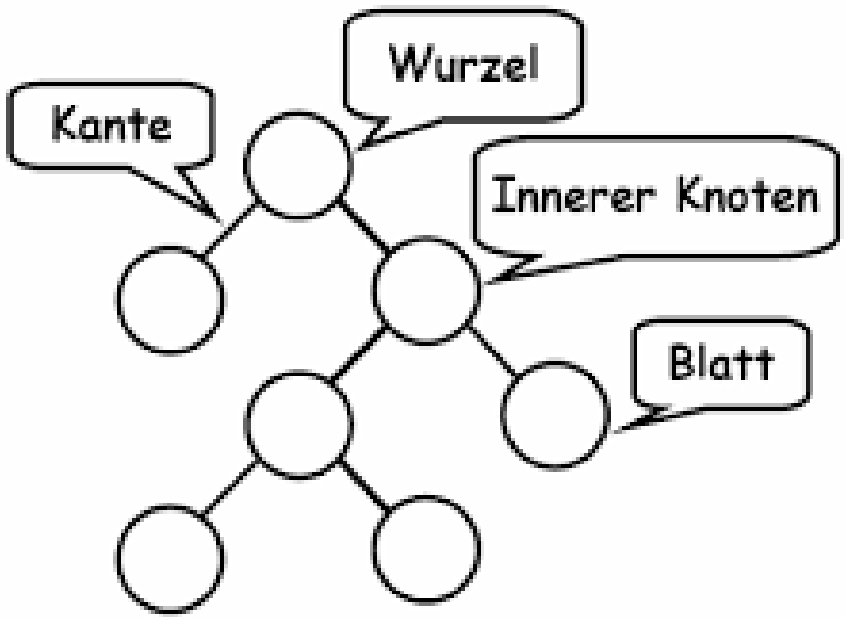
Speicherverbrauch:
Um Kollisionen zu vermeiden, reservieren Hash-Strukturen oft mehr Speicher, als sie aktuell belegen.

Kollisionen:
Wenn zwei verschiedene Schlüssel denselben Hash-Wert erzeugen, sinkt die Performance, da dieser Konflikt gelöst werden muss, z.B. mit Chaining (verkettete Liste, die alle Elemente mit demselben Hash-Wert nacheinander speichert – diese muss dann durchlaufen werden, was Zeit kostet)

Baum (Tree)

Definition: Hierarchische Struktur aus Knoten und Kanten.

Wichtig: *Ein* "Wurzel„(„Root“)-Knoten verzweigt sich immer weiter; keine Zyklen erlaubt.

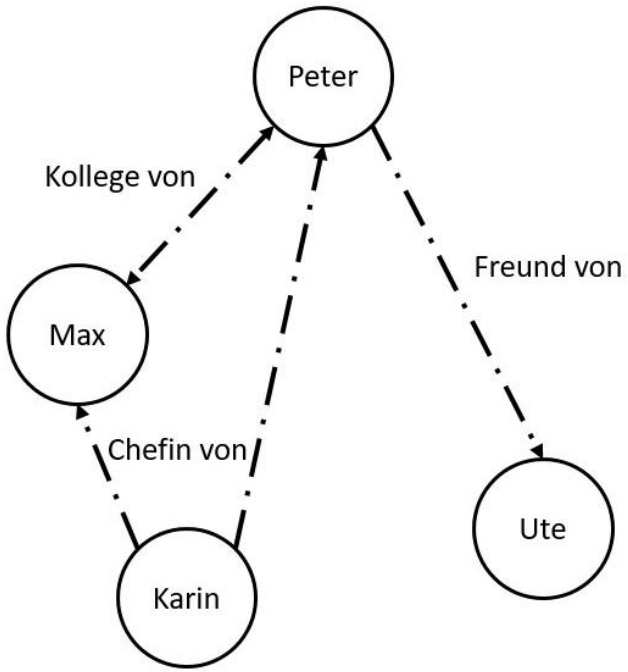


Bildquelle: Kapitel10_01.pptx

Graphen

Definition: Beliebig vernetzte Struktur aus Knoten und Kanten.

Wichtig: Ermöglicht Zyklen und komplexe Beziehungen (z. B. Routen).



Bildquelle: Graphdatenbanken Definition & Erklärung | Datenbank Lexikon

Überblick

Struktur	Definition / Merkmal	Wichtiger Vorteil / Eigenschaft
Array	Datenstruktur mit fester Länge.	Schneller Zugriff über den Index.
ArrayList	Dynamisch erweiterbare Liste.	Flexibel in der Größe (wächst mit).
Linked List	Verkettete Struktur aus Knoten.	Einfügen am Anfang besonders schnell.
Stack	Stapel-Speicher (LIFO-Prinzip).	Letztes Element wird zuerst entnommen.
Queue	Warteschlange (FIFO-Prinzip).	Erstes Element wird zuerst entnommen.
Set (Menge)	Ungeordnete Sammlung einmaliger Elemente.	Erlaubt keine Duplikate.
Hashbasiert	Speicherung via Schlüssel-Wert-Paaren.	Schnellster Zugriff unabhängig von der Größe.
Baum (Tree)	Hierarchische Struktur aus Knoten & Kanten.	Hat einen Root-Knoten und keine Zyklen.
Graph	Beliebig vernetzte Knoten & Kanten.	Erlaubt Zyklen (geschlossene Wege).